

NOW_WHAT?

ENGINE v1.2 / PRODUCT v1.1.0

Complete Functionality and Architecture Report

A local-first daily command system that turns context into a deterministic, explainable next action.

PLAN -> ACT -> LOG -> REFLECT -> NOW_WHAT?

Source snapshot: 3d31a4b on feature/needs-profile-data-model

Generated: 2026-07-12 / Asia-Dhaka

Local-first. No account. No required cloud. No AI/ML.

Document Control

This report is generated from the committed NOW_WHAT? source tree, not from a product roadmap or external specification. It describes implemented behavior at the source snapshot below and separates working capability from known limitations.

Attribute	Value
Repository branch	feature/needs-profile-data-model
Git commit	3d31a4b
Package version	1.1.0
Engine reference	v1.2 foundation and recommendation deck
Profile schema	3
Export format	3
Core packaged files	144
Source files under src/	127
Micro-action templates	54
Focused engine QA assertions	77

VERSION NOTE

The user-facing and package version remains 1.1.0. The term Engine v1.2 refers to the newer Needs Profile, state/energy scoring, micro-action library, recommendation deck, debug visibility, category normalization, and QA foundation now present in the codebase.

Table of Contents

Document Control 2

Table of Contents 3

1. Introducing NOW_WHAT? 7

- Why the product is differentiated . . . 7
- Who it is for . . . 7
- The operating loop . . . 7
- Capability snapshot . . . 8

2. Product Principles and Boundaries 9

- Hard product constraints embodied by the code . . . 9
- What local-first means here . . . 9
- What explainable means here . . . 9
- What the product intentionally does not claim . . . 9

3. System Architecture 10

- Technology stack . . . 10
- Layered architecture . . . 10
- Runtime bootstrap . . . 10
- Routes . . . 11
- Application event bus . . . 11

4. Local Identity and Data Core 12

- Stable local profile without accounts . . . 12
  - Profile resolution algorithm . . . 12
  - Legacy migration safety . . . 12
- Dexie schema versions . . . 12
- Profile-scoped stores . . . 13
- Meta database stores . . . 13

5. Domain Model Reference 14

- Task . . . 14
- Habit and HabitCompletion . . . 14
- Goal . . . 15
- CalendarBlock . . . 15
- ActivityLog . . . 15

DailyPlan . . . 16
Suggestion, RecommendationDeck, CandidateScoreBreakdown . . . 16
NeedsProfile . . . 17
DailyReview and WeeklyReview . . . 17
Reminder . . . 18
Settings and LocalProfile . . . 18

6. Feature Modules and User Workflows 19

Dashboard command surface . . . 19
Tasks . . . 19
Habits . . . 19
Goals . . . 19
Daily Plan . . . 19
Calendar Import Core . . . 20
ICS parser behavior . . . 20
Quick Log and Lifestyle Context . . . 20
Timeline and activity feed . . . 20
Daily and weekly review . . . 20
Reminders . . . 21
Tutorial and recommendation onboarding . . . 21

7. NOW_WHAT? Engine Architecture 22

Public engine entry points . . . 22
Context contract . . . 23
No-context gate . . . 23
The two energy concepts . . . 23
User Energy Level normalization . . . 24
State inference . . . 24
Focus capacity derivation . . . 24
Task readiness preselection . . . 24
Task state-fit adjustments . . . 25
Base candidate rule order . . . 25
Hard-rule semantics . . . 26
Micro-action eligibility . . . 26
eligibleWhen vocabulary . . . 26
Needs coverage . . . 27
Final v1.2 candidate score . . . 27
Urgency score . . . 27
Needs and priority-mode score . . . 27

Task energy fit matrix . . . 28
Additional user-energy fit . . . 28
Low-energy preference score . . . 28
Final state and time fit . . . 28
Cooldown and variety . . . 29
Task category normalization and scoring . . . 29
Recommendation deck construction . . . 29
Accept, dismiss, and alternative behavior . . . 30
Explanation and debug visibility . . . 30

8. Data Lifecycle, Export, Import, and Integrity 31
Write paths . . . 31
Export format v3 . . . 31
Backup validation . . . 31
Restore semantics . . . 31
Clear and demo behavior . . . 32
Integrity checks . . . 32

9. PWA and Offline Architecture 33
Manifest . . . 33
Service worker behavior . . . 33
Install prompt state . . . 33
Offline boundary . . . 33

10. Quality Assurance and Verification 34
Focused v1.2 engine harness . . . 34
Latest verification state . . . 34
Important test gaps . . . 34

11. Security, Privacy, and Reliability Posture 35
Privacy strengths . . . 35
Security and data-loss considerations . . . 35

12. Known Limitations and Technical Debt 36
Recommended extension order . . . 36

13. Architecture Extension Map 37
Safe module boundaries for future work . . . 37
Design rule for future engine additions . . . 37

Appendix A. Complete Micro-Action Library 38

Appendix B. Core File Inventory **41**

Appendix C. Package Integrity and Handoff Notes **45**

Source fingerprint 45

LLM handoff guidance 45

Glossary 45

1. Introducing NOW_WHAT?

NOW_WHAT? is a local-first personal command system built for the moment when a person has commitments, context, and limited attention, but still needs one concrete answer: what should happen next?

Most productivity software is optimized for capture, organization, or reporting. NOW_WHAT? adds a decision layer. It combines tasks, time, daily planning, goals, habits, calendar commitments, recent activity, user energy, current operational state, review status, and recommendation preferences, then resolves that context into a primary action with transparent reasoning and alternatives.

THE PRODUCT PROMISE

A private, explainable command surface that reduces next-action uncertainty without requiring an account, a cloud service, an AI model, a social graph, or a reward loop.

Why the product is differentiated

- Local-first by architecture: user records live in profile-specific IndexedDB databases on the current browser or device.

- Deterministic by design: recommendations are produced by explicit rules and numeric score components, not opaque model output.

- Context-aware without diagnosis: mood, state, focus, rest, distraction, and numeric user energy are operational signals, not clinical judgments.

- Action-oriented: the output is not another dashboard metric; it is a command with accept, dismiss, and alternative-selection paths.

- Neutral and non-gamified: the app avoids streaks, points, social comparison, shame, and motivational coaching.

- Portable: profile metadata and all twelve profile data stores are represented in a human-readable JSON export format.

Who it is for

The current system is useful for students, knowledge workers, independent builders, and people managing mixed work, health, household, and creative responsibilities. It is especially suited to users who value privacy, want lightweight structure, and prefer an explainable rule system over an AI assistant.

The operating loop

```
PLAN -> ACT -> LOG -> REFLECT -> NOW_WHAT?

Plan commitments and fixed time.
Act on one bounded item.
Log factual activity and current state.
Reflect through daily and weekly review.
Recompute the next action from updated context.
```

Capability snapshot

Capability	Implemented surface	System effect
Manual tasks	Priority, category, task energy, estimate, due date, repeat metadata, goal link, notes	Feeds urgency, time fit, energy fit, goals, reminders, plan, review, and NOW_WHAT?
Habits	Daily/weekly/custom, boolean/quantity/time/count, target, time window, goal link	Feeds habit windows, completion logs, reminders, daily plan, review, and recommendations
Goals	Category, horizon, status, task/habit links, derived progress	Active goal links influence task value and weekly progress notes
Calendar	Manual blocks, ICS import, dedupe, block-to-task conversion	Defines active commitments and the available window before the next block
Daily plan	Primary focus, planned tasks/habits/blocks, carry-over	Raises planned work and provides review context
Quick Log	Water, meal, focus, exercise, mood, energy, rest, distraction, notes	Updates lifestyle context, timeline, reviews, reminders, and engine state
Review	Daily facts, carry-over, missed work, weekly summaries and patterns	Supplies review candidates, reminders, and historical summaries
Reminders	System and manual reminders, quiet hours, optional browser notifications	Foreground scheduler creates and cancels action-specific reminders
NOW_WHAT?	Primary recommendation, two alternatives, cooldowns, explanations, debug	Resolves context into a deterministic recommendation deck
Data controls	Profile status, export, import, integrity checks, danger zone	Makes local data inspectable, portable, repairable, and resettable
PWA	Manifest, service worker, offline shell, install metadata	Supports installable and offline-capable web delivery

2. Product Principles and Boundaries

Hard product constraints embodied by the code

- No login or signup flow.
- No required backend or cloud connection.
- No AI or machine-learning inference in the recommendation engine.
- No social features, streaks, rewards, points, or public profiles.
- No therapy, diagnosis, or emotional coaching language.
- No automatic demo data on a normal first launch.
- Calendar is context, not a full calendar product; imported blocks remain blocks unless converted into tasks.
- The interface remains a strict black-and-white HUD-minimal command surface.

What local-first means here

Local-first is not merely a privacy statement in NOW_WHAT?. It is the primary persistence architecture. A metadata database resolves a stable local profile, then the application opens a profile-specific Dexie database and redirects the mutable database binding to that instance before rendering the routed application.

What explainable means here

Every recommendation is associated with a named rule, stable candidate identity, explicit context snapshot, component score breakdown, factual reason text, and suggestion lifecycle status. The hidden Shift+N debug panel exposes raw context, energy signals, profile choices, category effects, cooldown exclusions, hard-rule state, deck identities, and per-candidate scores.

What the product intentionally does not claim

NOW_WHAT? does not claim to optimize a person's life, infer mental health, predict behavior, or replace judgment. It ranks available actions from locally recorded facts and user-configured preferences. Critical deadlines and active calendar commitments retain precedence over convenience-oriented fit adjustments.

3. System Architecture

Technology stack

Layer	Technology	Role
Application UI	React 19.1 + React DOM	Component tree, hooks, modals, routes, live updates
Build tool	Vite 7.3 + @vitejs/plugin-react	Development server and production bundling
Language	TypeScript 6.0	Shared domain contracts and compile-time checks
Routing	Wouter 3.3.5	Client-side routes under the configured Vite base
Persistence	Dexie 4.4.4	Typed IndexedDB schema, indexes, transactions, and migration versions
Reactive persistence	dexie-react-hooks	useLiveQuery-backed UI synchronization
Styling	Global CSS variables plus scoped inline styles	Black-and-white HUD identity and responsive behavior
PWA	Web manifest + custom service worker	Install metadata, shell caching, offline navigation fallback
QA	Vite SSR harness + node:assert	Deterministic engine, import, integrity, and debug assertions

Layered architecture

COMMAND SURFACE	React routes, dashboard, modals, keyboard shortcuts, recommendation deck
FEATURE MODULES	Tasks, habits, goals, calendar, daily plan, quick log, timeline, review, reminders
DECISION LAYER	Context builder, rule candidates, state fit, v1.2 scoring, cooldowns, micro-actions
DATA SERVICES	Profile bootstrap, calendar parser, reminders, reviews, export/import, integrity
LOCAL STORAGE	nowwhat_meta plus profile-specific Dexie database in browser IndexedDB
OFFLINE SHELL	Vite build, web manifest, service worker cache, local static assets

Figure 1. The application is web-first, but its behavior is divided into command-surface, feature, decision, data-service, storage, and offline-shell layers.

Runtime bootstrap

- main.tsx mounts App into the root element and loads the global stylesheet.

- App registers the service worker only in production; development unregisters existing workers.

- initializeProfileSession opens nowwhat_meta, resolves or creates a LocalProfile, creates a profile-specific database, swaps the active db binding, migrates eligible legacy data once, ensures Settings, and ensures the default NeedsProfile.

- Routes and global modals are not rendered until profile initialization succeeds.

- After profile boot, the reminder scheduler runs immediately, every five minutes, and whenever the recommendation-regeneration event fires.

- The NOW_WHAT? card generates on mount, every five minutes, on focus/visibility return, and on explicit regenerate events.

Routes

Route	Surface	Primary purpose
`/`	Dashboard	Recommendation deck, daily plan, reminders, calendar/lifestyle/goal strips, activity
`/tasks`	Tasks	Task list, completion, delete, and creation
`/habits`	Habits	Habit filters, completion controls, create/edit/delete
`/goals`	Goals	Goal status filters, derived progress, links, create/edit/delete
`/calendar`	Calendar	Today/upcoming/past blocks, manual CRUD, ICS import, block conversion
`/review`	Review	Daily and weekly review plus carry-over actions
`/reminders`	Reminders	Manual reminders, lifecycle lists, settings and permission opt-in
`/timeline`	Timeline	Chronological activity log
`/data`	Data	Status, recommendation profile, export, import, integrity, danger zone
`/guide`	Guide	On-device factual product tutorial

Application event bus

The code uses browser CustomEvent/Event messages as a lightweight cross-module command bus. This avoids deep prop wiring, but it also means event names are informal contracts rather than typed interfaces.

Event	Producer examples	Consumer/result
`regenerate_now_what`	Task/habit/log/calendar writes, reminder actions	NOW_WHAT? regenerates and reminder scheduler runs
`open_task_modal`	Dashboard, micro-action	App opens TaskModal
`open_habit_modal`	Dashboard, micro-action	App opens HabitModal
`open_goal_modal`	Shortcut G, goal commands	App opens GoalModal
`open_calendar_block_modal`	Dashboard, micro-action	App opens CalendarBlockModal
`open_quick_log`	Shortcut L, recommendations, reminders	App opens QuickLogModal
`navigate_tasks/habits/calendar/review/dashboard`	Recommendation actions	NavigationHandler changes route
`open_recommendation_profile`	Micro-action	Selects Data/Profile and navigates
`engine_debug_update`	NowWhatCard generation and alternative selection	App refreshes hidden EngineDebugPanel data
`show_toast`	Feature operations	ToastProvider displays factual status

4. Local Identity and Data Core

Stable local profile without accounts

The meta database nowwhat_meta contains a profiles table and an appMeta key/value table. On first boot the app generates an identifier in the form nwloc_<UUID>, assigns the display name Local Profile, and creates the profile database name nowwhat_profile_<profileId>. The active profile ID is persisted under appMeta.activeProfileId.

Profile resolution algorithm

```
activeId = appMeta['activeProfileId']
profile = profiles.get(activeId)
if missing:
    profile = first profile by createdAt OR create LocalProfile
    persist activeProfileId
refresh lastOpenedAt, updatedAt, schemaVersion, and dbName
open nowwhat_profile_<profileId>
setActiveDb(profileDb)
run one-time legacy migration
ensure Settings and default NeedsProfile
```

Legacy migration safety

- Migration is tracked per profile with an appMeta key named legacyMigration:<profileId>.
- Legacy copying is skipped if the target profile already contains any records.
- The legacy nowwhat_db database is copied only when it exists and contains data.
- All twelve profile stores are copied inside one Dexie read/write transaction.
- The legacy database is closed after inspection or copying; the source is not deleted.

Dexie schema versions

Version	Change	Stores/indexes
1	Base application schema	settings, tasks, habits, goals, calendarBlocks, activityLogs, dailyPlans, suggestions, dailyReviews, weeklyReviews
2	Reminder subsystem	Adds reminders indexed by id, type, status, scheduledAt, targetId, targetType, createdAt
3	Recommendation profile	Adds needsProfiles indexed by id

Profile-scoped stores

Store	Primary indexes	Ownership and role
settings	id	One effective settings record for version, tutorial, reminder, and localProfileId metadata
tasks	id, status, priority, dueDate, energyRequired, goalId, createdAt	Manual commitments and completion state
habits	id, frequency, createdAt	Repeated actions and embedded completion history
goals	id, status, horizon, createdAt	Longer-horizon containers and bidirectional task/habit links
calendarBlocks	id, startAt, endAt, createdAt	Manual/imported fixed-time context and conversion references
activityLogs	id, type, createdAt, relatedTaskId, relatedHabitId	Factual timeline and lifestyle/state signals
dailyPlans	id, date, createdAt	Daily focus, planned IDs, block IDs, carry-over, review status
suggestions	id, status, actionType, createdAt	Recommendation lifecycle, cooldown source, debug snapshot
dailyReviews	id, date, createdAt	Generated daily facts, note, and completion state
weeklyReviews	id, weekStart, createdAt	Generated weekly aggregates and observed patterns
reminders	id, type, status, scheduledAt, targetId, targetType, createdAt	System/manual reminder lifecycle
needsProfiles	id	Recommendation needs, priority mode, low-energy mode, State Check schedule

Meta database stores

Store	Indexes	Role
profiles	id, dbName, createdAt, lastOpenedAt	Stable local identity records and profile database names
appMeta	key	activeProfileId and one-time legacy migration markers

ARCHITECTURAL TRADEOFF

Profile isolation is achieved per database, so normal records do not need profileId columns. The application still exposes a mutable global db binding that feature components import directly. Profile bootstrap swaps that binding once before rendering; a future multi-profile switcher would need a deliberate remount/subscription strategy.

5. Domain Model Reference

The following tables document the fields that cross module boundaries. Optional fields preserve backward compatibility with older records and imported backups. IDs are generated with crypto.randomUUID unless otherwise noted.

Task

A one-time or repeat-metadata commitment used by planning, goals, reminders, review, and the recommendation engine.

Field	Type	Operational role
id	string	Primary key.
title	string	User-visible action name and optional primary-focus text match.
category	TaskCategory/string	Semantic domain normalized for Needs Profile, admin, creative, and health fit.
priority	low|medium|high|critical	Urgency score, candidate rules, reminder override, and tie-breaking input.
status	pending|complete|skipped|missed	Pending eligibility and completion lifecycle.
estimatedMinutes	number?	Short/long classification and available-window fit.
dueDate	YYYY-MM-DD?	Overdue/today/future urgency and reminder generation.
dueTime	HH:MM?	Deadline timestamp and four-hour hard-deadline detection.
repeatRule	RepeatRule	none/daily/weekly/custom metadata; future-instance generation is not implemented.
energyRequired	low|medium|high	Task effort requirement, distinct from numeric user Energy Level.
goalId	string?	Link to an active Goal; raises task value and weekly progress notes.
notes	string?	Free detail; calendar-converted tasks include source-block provenance.
carryOverHistory	string[]	Dates on which the task was carried or marked missed.
missedCount	number	Neutral recurrence/carry-over history signal and critical reason text.
createdAt/updatedAt	ISO string	Creation ordering, recency, auditing.
completedAt	ISO string?	Completion timestamp.

Habit and HabitCompletion

A repeated action with embedded completion history and optional quantitative tracking.

Field	Type	Operational role
Habit.id	string	Primary key.
name	string	Visible label and recommendation target.
category	string	Grouping/review metadata; not currently a NOW_WHAT? scoring input.
frequency	daily|weekly|custom	Daily habits feed engine windows and review calculations.
completionType	boolean|quantity|time|count	Controls completion UI and recorded value.
targetValue/targetUnit	number?/string?	Expected measurement and display unit.
timeWindowStart/End	HH:MM?	Engine eligibility and reminder closing-window logic.
minimumVersion	string?	Optional minimum behavior label.
goalId	string?	Bidirectional Goal link.
notes	string?	User detail.

Field	Type	Operational role
completionHistory	HabitCompletion[]	Date, optional numeric value, and completion timestamp.
createdAt/updatedAt	ISO string	Lifecycle metadata.

Goal

A longer-horizon container whose progress is derived from linked work rather than manually asserted.

Field	Type	Operational role
id	string	Primary key.
title	string	Visible goal name.
category	string	Grouping/review metadata; not a direct engine score input.
horizon	week|month|semester|year|ongoing	Planning horizon shown in the goal surface.
linkedTaskIds	string[]	Bidirectional links reconciled transactionally by GoalModal.
linkedHabitIds	string[]	Bidirectional links reconciled transactionally by GoalModal.
status	active|paused|complete	Only active goals contribute to current context and task value.
reviewNotes	string?	Goal-level factual notes.
createdAt/updatedAt	ISO string	Lifecycle metadata.

CalendarBlock

Fixed-time context imported from ICS or entered manually. A block is not a task unless explicitly converted.

Field	Type	Operational role
id	string	Primary key.
source	manual|ics|google_ics|google_api|apple_ics|google_stub|apple_stub	Origin marker and dedupe namespace.
externalId	string?	ICS UID or external identifier; preferred dedupe key.
title	string	Commitment label and converted task title.
startAt/endAt	local datetime string	Active-block detection, sorting, available window, reminders.
location	string?	Imported/manual place detail.
calendarName	string?	Optional source calendar label.
importedAt	ISO string?	Import provenance.
convertedTaskIds	string[]	Idempotent references to task conversions.
createdAt/updatedAt	ISO string	Lifecycle metadata.

ActivityLog

The append-oriented factual event stream used by Timeline, lifestyle context, reviews, reminders, and the engine.

Field	Type	Operational role
id	string	Primary key.
type	13-value LogType	task_completed, habit_completed, water, meal, exercise, focus, mood, energy, state_check, rest, distraction, calendar, or note.

Field	Type	Operational role
label	string	Human-readable factual description.
startAt/endAt	ISO string?	Optional interval metadata.
durationMinutes	number?	Focus, exercise, rest, and distraction aggregation.
quantity/unit	number?/string?	Water and 1-5 mood/energy values or other measurements.
state	StateType?	Neutral operational state captured by State Check.
energyLevel	1|2|3|4|5?	Required numeric user energy for State Check.
relatedTaskId	string?	Task completion/reference link.
relatedHabitId	string?	Habit completion/reference link.
relatedGoalId	string?	Optional goal reference.
notes	string?	Optional factual detail.
createdAt	ISO string	Timeline and latest-signal ordering.

DailyPlan

The per-date operating plan that turns the broader database into today's intended context.

Field	Type	Operational role
id	string	Primary key.
date	YYYY-MM-DD	One effective plan per local date.
primaryFocus	string?	Current focus and daily_focus matching text.
plannedTaskIds	string[]	Planned task boosts and review denominator.
plannedHabitIds	string[]	Planned habit context.
calendarBlockIds	string[]	Blocks included in the plan/review.
lifestyleMinimums	object	Optional water, meals, movement, focus, and rest targets.
carryOverTaskIds	string[]	Neutral carry-over list and review exclusion from missed work.
reviewStatus	not_started|in_progress|complete	Dashboard and review lifecycle.
createdAt/updatedAt	ISO string	Lifecycle metadata.

Suggestion, RecommendationDeck, CandidateScoreBreakdown

The persisted and inspectable output contracts of the NOW_WHAT? engine.

Field	Type	Operational role
Suggestion.id	string	Primary key.
actionType	SuggestionActionType	start_task, complete_habit, lifestyle/review/plan/block actions, clear_state, or micro_action.
targetId	string?	Task, habit, block, or other domain target.
command/targetLabel/reason	string	Visible action, object, and factual explanation.
status	shown|accepted|dismissed|completed|ignored	Lifecycle and cooldown source.
contextSnapshot	Record<string, unknown>	Profile, context, identity, score, cooldown, and action-intent audit data.
Deck.primary	Suggestion?	Highest-ranked available candidate.

Field	Type	Operational role
Deck.alternatives	Suggestion[]	Up to two alternatives, initially action-type diverse.
Deck.state	ready|no_context|cooldown|empty	Controls command-surface behavior.
Deck.supportText	string?	Critical-task context retained when another action is primary.
Deck.debug	CandidateScoreBreakdown[]?	Per-candidate scoring audit.
Score fields	numeric components	base, urgency, needs, user energy, task energy, state, time, cooldown, variety, final.

NeedsProfile

A singleton per profile that personalizes deterministic ranking and State Check timing.

Field	Type	Operational role
id	'default'	Singleton primary key.
selectedNeeds	NeedType[]	school_work, habit_consistency, health_energy, life_admin, creative_projects.
priorityMode	5-value mode	deadline, critical, easy_start, daily_focus, or energy_match scoring policy.
lowEnergyPreference	5-value mode	short_tasks, admin, recovery, health, or critical_only.
checkInEnabled	boolean	Enables scheduled State Check prompts.
checkInIntervalHours	integer 1-168	Base State Check interval.
quietHoursStart/End	HH:MM	State Check suppression window.
onboarding timestamps	ISO string?	Completed or dismissed recommendation-profile onboarding.
lastStateCheckAt/nextStateCheckAt	ISO string?	Schedule state.
stateCheckDismissCount	integer?	Repeated dismissal backoff counter.
createdAt/updatedAt	ISO string	Lifecycle metadata.

DailyReview and WeeklyReview

Generated factual summaries with optional user reflection and observed patterns.

Field	Type	Operational role
DailyReview.date	YYYY-MM-DD	Reviewed day.
task/habit/block ID arrays	string[]?	Completed, missed, carried, habit, and calendar references.
daily metrics	number?	Focus, distraction, rest, water, meals, movement, sleep.
generatedSummary	string?	Rule-generated factual narrative.
reflection/reflectionNote	string?	Legacy and current user note fields.
status	draft|complete?	Review completion gate.
WeeklyReview.weekStart/weekEnd	YYYY-MM-DD	Monday-Sunday range.
weekly metrics	number	Task/habit counts and lifestyle totals.
pattern arrays	string[]	Consistent/missed habits, repeated missed tasks, goal notes.
bestFocusWindow/struggleWindow	string?	Two-hour windows generated only from at least three relevant logs.

Reminder

A system- or user-authored scheduled action with in-app and optional browser delivery.

Field	Type	Operational role
id	string	Primary key.
type	10-value type	task_due, habit_window, calendar_block, planned_focus, daily_review, previous_review, hydration, movement, manual, custom.
title/message	string	Visible factual copy.
targetType/targetId	typed strings?	Navigation/completion target.
scheduledAt	ISO string	Due determination and sorting.
status	scheduled|sent|dismissed|completed|cancelled	Lifecycle.
source	system|user	Origin.
repeat	none|daily|weekly?	Stored recurrence metadata; manual reminders use none.
enabled	boolean	Due filtering.
createdAt/updatedAt/sentAt	ISO string	Lifecycle timestamps.

Settings and LocalProfile

Configuration metadata split between the profile data database and the meta database.

Field	Type	Operational role
Settings.theme	'dark'	Current fixed visual mode.
appVersion/schemaVersion/localProfileId	optional metadata	Version and profile ownership markers.
lastExportAt/lastImportAt	ISO string?	Backup activity metadata.
tutorial timestamps	ISO string?	Prompt seen, completed, dismissed state.
reminder settings	boolean/time fields	Global enable, notification opt-in, quiet hours, per-type toggles.
LocalProfile.id	nwloc_<UUID>	Stable browser/device profile identity.
displayName/deviceLabel	string	Exportable profile metadata.
dbName	string	Profile-specific IndexedDB database name.
schemaVersion	number	Current profile schema metadata.
createdAt/updatedAt/lastOpenedAt	ISO string	Profile lifecycle.

6. Feature Modules and User Workflows

Dashboard command surface

Dashboard composes the recommendation deck, review prompt, in-app reminder tray, Daily Plan, calendar strip, lifestyle strip, active-goal strip, and recent activity feed inside a restrained 960-pixel content region. It is the first screen, not a marketing page.

Tasks

- TaskModal creates tasks with centralized category options, priority, task energy requirement, estimate, due date, repeat metadata, goal link, and notes.

- TasksPage filters by status and uses TaskRow for completion and deletion.

- Completion creates a task_completed ActivityLog, timestamps the task, and regenerates recommendations.

- The engine distinguishes numeric user Energy Level 1-5 from Task.energyRequired low/medium/high.

- RepeatRule is stored, but there is no service that materializes future recurring task instances in the current code.

Habits

- Create/edit supports eight shared standard categories plus an explicit OTHER / CUSTOM path; legacy custom values remain visible and preserved.

- Completion types are boolean, quantity, time, and count; non-boolean modes expose contextual target labels and units.

- Daily incomplete habits feed NOW_WHAT?. Time-windowed habits become hard candidates when their window closes within 30 minutes.

- Completion appends HabitCompletion and a habit_completed ActivityLog.

- Habit category is for grouping/review only; frequency, completion history, time window, and plan status are the engine-relevant fields.

Goals

- Goals have independent category, horizon, and status controls. Category is not conflated with horizon or status.

- GoalModal reconciles task.goalId and habit.goalId with the goal's linked ID arrays inside a Dexie transaction.

- Only active goals enter current context. A task linked to an active goal receives readiness and high-energy fit benefits.

- Goal progress is derived from linked task/habit state; there is no manual gamified progress slider.

- Goal category itself does not affect NOW_WHAT? scoring.

Daily Plan

- getOrCreateTodaysPlan creates one local-date record with empty arrays and not_started review status.

- Primary focus, planned tasks, planned habits, and calendar blocks are mutable through focused helper functions.

- Carry-over adds the task to planned and carry-over arrays, records the source date, and increments missedCount only once per date.

- Planned work affects task readiness, daily_focus mode, high user-energy fit, review summaries, and no-context detection.

Calendar Import Core

Calendar blocks are treated as read-only time context until explicitly converted. Manual blocks can be edited/deleted; imported blocks are normalized and deduplicated. The user-facing surface separates today, fourteen-day upcoming, optional past, ICS import, and future connector placeholders.

ICS parser behavior

- Unfolds folded content lines and parses VEVENT blocks.

- Reads UID, SUMMARY, DTSTART, DTEND, LOCATION, and RRULE.

- Unescapes newline, comma, semicolon, and backslash sequences.

- Supports date-only values, local/floating date-times, TZID-tagged wall time, and UTC Z values converted to local wall time.

- Does not perform VTIMEZONE conversion or recurrence expansion; RRULE events are imported once and counted as unsupported recurrence.

- Deduplication prefers source plus externalId; otherwise it uses source, title, startAt, and endAt.

- Block-to-task conversion is idempotent while a previously converted task still exists and records the task ID on the block.

Quick Log and Lifestyle Context

Action	Stored fields	Downstream use
Water +250/+500	quantity ml	Hydration totals, last-water age, reminders, reviews, lifestyle strip
Focus	durationMinutes and notes	Focus total, last-focus age, focus capacity, reviews
Meal	label and notes	Meal count and review
Exercise	label and durationMinutes	Movement totals, reminders, reviews, restless actions
Mood	integer quantity 1-5 and notes	Today-state inference; invalid values are rejected/ignored
Energy	integer quantity 1-5 and notes	Latest user energy band/signal and task fit
Rest	durationMinutes and notes	Recovery capacity, state inference, reviews
Distraction	durationMinutes and notes	Restless signal, reviews, weekly struggle window
Free note	note label	Timeline and reviewable activity

Timeline and activity feed

Timeline is a factual chronological view over ActivityLog. Dashboard ActivityFeed provides a smaller recent slice. Activity records are intentionally generic enough to aggregate lifestyle data while retaining related task, habit, or goal identifiers where applicable.

Daily and weekly review

- Review gating requires reviewable activity or a task created from a calendar block; an empty day does not produce a false review prompt.

- Daily generation aggregates completed IDs, focus, distraction, rest, water, meals, movement, plan membership, carry-over, and missed planned work.

- Completing a daily review also marks the day's DailyPlan reviewStatus complete.

- Carry-to-tomorrow uses the reviewed date as the source date, so reviewing yesterday correctly carries work into today.

- Weekly review aggregates Monday-Sunday logs, repeated missed tasks, habit consistency, active-goal completions, and two-hour focus/distraction windows when at least three logs exist.

Reminders

Reminder	Trigger	Quiet-hours behavior
Task due	Pending task overdue or due today	Suppressed except critical overdue tasks
Habit window	Daily incomplete habit, window open, closes in 0-30 minutes	Suppressed
Calendar block	Today's block starts in 0-20 minutes	Suppressed
Daily review	After 20:00, review incomplete, reviewable activity exists	Suppressed
Previous review	Yesterday has reviewable activity and incomplete review	Suppressed
Hydration	Opt-in, after 08:00, no water for at least 180 minutes	Suppressed; three-hour active-reminder dedupe
Movement	Opt-in, after 14:00, no exercise log today	Suppressed
Manual	User title, date, time, optional note	Stored as scheduled; no repeat UI

The scheduler uses a module-level single-flight lock, cancels reminders when target work is completed, deduplicates by type/target/local date, and removes terminal reminders older than seven days. The in-app tray marks scheduled reminders sent when due, can log 250 ml water or complete a habit inline, and can route to the relevant module. Browser notifications require explicit permission and are silent.

Tutorial and recommendation onboarding

- Tutorial status is stored in Settings with seen, completed, and dismissed timestamps.
- The Guide contains seventeen factual sections and the keyboard shortcuts.
- Recommendation Profile onboarding waits until tutorial handling is complete and can be configured or dismissed.
- Legacy/data-bearing imports suppress false onboarding when old backups do not contain modern onboarding metadata.

7. NOW_WHAT? Engine Architecture

The engine is a deterministic pipeline rather than a single priority list. It first builds a profile-scoped context, creates rule and micro-action candidates, applies state-aware preselection, filters cooldowns, scores every candidate, builds a recommendation deck, persists the primary suggestion, and publishes a debug snapshot.

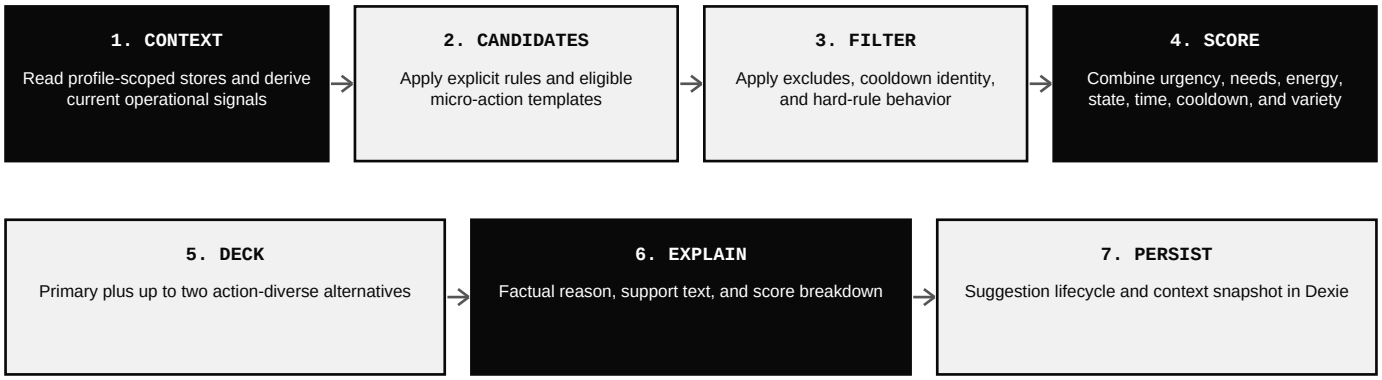


Figure 2. Recommendation generation is explicit at every stage and produces inspectable state.

Public engine entry points

Function	Responsibility
`getCurrentContext`	Reads profile-scoped data and derives all Context signals.
`getCandidateActions`	Creates base rule candidates, eligible micro-actions, state fallback, and stable identities.
`scoreCandidate`	Calculates a complete CandidateScoreBreakdown and explanation additions.
`scoreAndRankCandidates`	Sorts by final score, then priority, then identity.
`generateRecommendationDeckWithDebugInfo`	Runs context, cooldown, scoring, primary/alternatives, persistence, and debug assembly.
`generateWithDebugInfo`	Compatibility wrapper returning primary suggestion, context, and candidates.
`generateNowWhatSuggestion`	Returns only the generated primary suggestion.
`acceptSuggestion/dismissSuggestion/completeSuggestion`	Updates suggestion lifecycle and cooldown metadata.
`persistAlternativeSuggestion`	Promotes an alternative and retires previously shown suggestions.

Context contract

Context group	Fields	Derivation
Clock/date	`now, hour, todayStr, yesterdayStr`	Current local clock and local-date helpers
Tasks	`pendingTasks, criticalOverdueTasks, tasksWithDeadlineToday, highPriorityTasks, completedTasksToday`	Task store plus today's task_completed logs
Activity	`meaningfulActivityCountToday`	Today's logs excluding calendar
Habits	`pendingHabitsInWindow, pendingHabitsAny`	Daily habits not completed today; optional current time window
Lifestyle	`waterMlToday, focus/rest/movement/distraction minutes, lastWaterAt, minutesSinceLastWater`	Today's logs plus latest water history
User state	`lastEnergyValue, userEnergyBand, lastMoodValue, latestMoodState, latestEnergyLevel, currentFocusCapacity`	Validated logs and deterministic normalizers
Focus recency	`lastFocusEndAt, minutesSinceLastFocus`	Latest focus log across history
Goals	`activeGoals, goalLinkedPendingTasks`	Active goal store and task.goalId
Daily plan	`todayDailyPlan, plannedTaskIds, plannedHabitIds, primaryFocus`	Plan matching today's local date
Calendar	`todaysCalendarBlocks, activeCalendarBlock, nextCalendarBlock, minutesUntilNextBlock, availableWindowMinutes`	Local-date blocks and current time; default window 480 minutes
Inventory	`totalTaskCount, totalHabitCount, totalCalendarBlockCount`	All records, not only pending/today
Context gate	`hasUserContext`	Any tasks, habits, blocks, active goals, plan content, meaningful activity, or review work
Review	`todayReviewStatus, todayReviewAvailable, hasReviewableActivityToday, yesterdayReviewIncomplete`	Review store and reviewable activity helpers
Profile	`needsProfile`	Default singleton, created during profile bootstrap
Recency/cooldown/debug	`minutesSinceLastTaskCreated, activeCooldownCount, selectedCandidateIdentity, excludedCandidateIdentities`	Task timestamps and generation pipeline

No-context gate

NeedsProfile settings alone do not count as actionable context. A fresh profile with no tasks, habits, calendar blocks, active goals, plan content, meaningful logs, or review work returns one NO CONTEXT AVAILABLE state candidate. This prevents hydration, review, or setup suggestions from appearing as if the user had already established a working context.

The two energy concepts

Concept	Storage	Values	Purpose
User Energy Level	ActivityLog.quantity for energy or ActivityLog.energyLevel for state_check	`Integer 1-5`	Current operating capacity and scoring band
Task energy requirement	Task.energyRequired	`low|medium|high`	Estimated effort demanded by a task

ENERGY INVARIANT

Category normalization and user Energy Level never overwrite Task.energyRequired. The QA harness explicitly checks that category changes leave the task-energy fit matrix unchanged.

User Energy Level normalization

Numeric value	EnergySignal	UserEnergyBand
1	LOW	low
2	LOW	low
3	MEDIUM	neutral
4	HIGH	high
5	HIGH	high

State inference

Signals	Derived state
Mood <=2 and energy <=2	OVERLOADED
Mood <=2 and energy >=3	STRESSED
Mood >=4 and energy >=4	CLEAR
Mood >=4 and energy <=2	RESTLESS if distraction >=20 and rest <10; otherwise LOW
No matching pair, energy LOW	LOW
No matching pair, mood <=2	STRESSED
Distraction >=20 and rest <10	RESTLESS
Otherwise	NEUTRAL

A valid explicit State Check state supersedes inferred state when no valid mood is logged today or when the State Check is at least as recent as today's latest valid mood log. Invalid 1-5 logs are filtered from context.

Focus capacity derivation

Capacity	Condition
RECOVERY	State OVERLOADED; or focus today >=90 with rest <10; or focus today >=60, last focus <20 minutes ago, and rest <10
LIGHT	State LOW or STRESSED; or RESTLESS with distraction >=20
DEEP	State CLEAR and EnergySignal HIGH
NORMAL	Default, including RESTLESS without the high-distraction condition

Task readiness preselection

Candidate rules such as high-priority, low-energy, deadline tie-break, and general pending task select a specific task using taskReadinessScore before final candidate scoring.

```
taskReadinessScore = taskBaseScore + taskStateFit.adjustment

taskBaseScore:
  priority: critical +5, high +3, medium +2, low +1
  overdue +5; due today +4; future due +1.5
  planned today +2
  goal linked +1.25
  fits available window +0.5; otherwise -2.5
  estimate missing -0.35

Tie break: lower estimated minutes, then earlier createdAt.
```

Task state-fit adjustments

State/capacity	Positive adjustments	Negative adjustments
CLEAR	planned +1.6; goal-linked +1.4; high/critical +1; >=45 min fitting +0.6; DEEP plus high task energy +0.5	None
LOW	task energy low +2; <=30 min +1.6; life-admin group +1.1	>=60 min high-energy nonurgent -2.4; other high-energy nonurgent -1
STRESSED	<=30 min +1.7; due/overdue +1; planned with estimate +1	missing estimate nonurgent -1.2; >=60 min nonurgent -1.5
RESTLESS	health-energy group +2.4; <=30 min +1; <=20 min +0.6	>=60 min nonurgent -1
OVERLOADED	smallest planned +2.6; other planned +1; low task energy +1.6; <=30 min +1.6	>=60 min nonurgent -2.6; high task energy nonurgent -1.2
RECOVERY capacity	short or low task energy +0.9	long plus high task energy nonurgent -1.8
LIGHT capacity	short or low task energy +0.6	long plus high task energy nonurgent -1.1
DEEP capacity	planned, goal-linked, or high priority and fits window +0.7	None

State explanations are appended only when the preselection adjustment magnitude reaches the visible threshold. Hard-rule candidates do not receive priority shifts.

Base candidate rule order

Base priority	Rule	Condition	Action
0	`active_calendar_block`	Current time lies inside a today's block	IN BLOCK / acknowledge_block
1	`critical_overdue_task`	First critical task with dueDate before today	START TASK
2	`closest_deadline_task`	Pending tasks have dueDate; sort by date, priority, readiness	START TASK
3	`high_priority_task`	Fitting high/critical task selected by readiness	START TASK
3.5	`end_of_day_all_done`	After 20:00, review available, all planned work done	REVIEW DAY
4	`easy_task_low_energy`	LOW signal/state or OVERLOADED; task is low-energy, short, or life-admin	START TASK
5	`habit_in_window`	Daily incomplete habit currently in window	COMPLETE HABIT
6	`hydration`	No water today or last water >180 minutes	LOG WATER
7	`rest_after_focus`	Focus exists; last focus >90 minutes ago; rest today <10	TAKE REST
8	`movement_midday`	After 12:00 and no movement today	LOG MOVEMENT
9	`general_pending_task`	Any pending task; prefer window-fitting then readiness	START TASK
10	`remaining_daily_habit`	Any incomplete daily habit	COMPLETE HABIT
10.5	`micro_<category>`	Template passes all eligibility gates and no hard rule exists	Template command / micro_action
11	`end_of_day_review`	After 20:00 with review available and incomplete	REVIEW DAY
11.5	`yesterday_review_incomplete`	Yesterday has reviewable activity and incomplete review	COMPLETE REVIEW
12	`clear_state`	Always appended when context exists	CLEAR STATE

Hard-rule semantics

Candidate	Hard score	Additional behavior
Active calendar block	`+1000`	Cooldown override; hard primary has no alternatives; suppresses micro-actions
Critical overdue task	`+900`	Cooldown override; hard primary has no alternatives; suppresses micro-actions
Closest deadline	`+800`	Hard when due today/overdue or timestamp is within four hours; suppresses micro-actions
Habit in window	`+700`	Hard when window closes in 0-30 minutes; suppresses micro-actions

Only active_calendar_block and critical_overdue_task explicitly bypass cooldown filtering. Closest-deadline and closing-habit candidates are hard for scoring/alternatives but can still be excluded by a matching active dismissal cooldown.

Micro-action eligibility

- The library contains 54 immutable templates across setup, planning, low_energy, recovery, movement, admin, focus, creative, review, and maintenance categories.

- Each template declares needsTags, stateTags, capacityTags, energyBandTags, eligibleWhen conditions, actionIntent, and cooldownHours.

- A template must pass target exclusion, state, focus capacity, user energy band, and every eligibleWhen condition.

- When selectedNeeds is nonempty, the template must share at least one selected need. With no selected needs, only setup, planning, maintenance, and review templates are admitted.

- With lowEnergyPreference critical_only and pending tasks, recovery, movement, and maintenance templates are filtered out.

- Any hard-rule candidate suppresses the entire micro-action pool for that generation.

eligibleWhen vocabulary

Condition	Meaning
`always`	No additional data condition
`user_context`	hasUserContext is true
`no_tasks/no_habits/no_calendar_blocks`	Corresponding total count is zero
`needs_profile_incomplete`	Recommendation onboarding is not completed
`pending_tasks/planned_tasks`	Pending work exists / a planned ID still points to a pending task
`active_habit_window`	At least one current incomplete daily habit window
`review_available`	Today's review is available or yesterday is incomplete
`goals_available/calendar_available`	At least one active goal / any calendar block
`activity_exists`	Meaningful activity exists today
`no_recent_task`	No task created within six hours
`no_urgent_work`	No critical or due/overdue pending task
`no_urgent_block`	No active block and at least ten minutes available

Needs coverage

Need	Templates tagged
school_work	38
habit_consistency	24
health_energy	26
life_admin	36
creative_projects	31

Final v1.2 candidate score

```
finalScore = baseScore
           + urgencyScore
           + needsScore
           + userEnergyFitScore
           + taskEnergyFitScore
           + stateFitScore
           + timeFitScore
           + cooldownPenalty
           + varietyPenalty

baseScore = max(0, 14 - basePriority) * 5 + hardRuleScore
```

Candidates sort by finalScore descending. Exact ties use adjusted candidate priority ascending, then stable identity lexicographically.

Urgency score

Signal	Adjustment
Task priority critical/high/medium	+24 / +13 / +5
Task overdue	+34
Task due today	+28
Due within <=1 / <=3 / <=7 days	+18 / +10 / +5
habit_in_window rule	+12
yesterday_review_incomplete rule	+5

Needs and priority-mode score

Input	Adjustment
Each selected need matching candidate tags	+7
deadline mode: due/overdue vs future due	+14 / +8
critical mode: critical vs high task	+14 / +8
easy_start mode: task <=30 minutes	+10
easy_start mode: low_energy micro-action	+7
daily_focus mode: planned task	+12
daily_focus mode: title includes primaryFocus	+7
energy_match mode with logged energy band	+5

Task energy fit matrix

User energy	Task low	Task medium	Task high
1	+14	-7	-20
2	+10	+3	-11
3	+1	+4	0
4	0	+5	+8
5	0	+5	+14

- At user energy 1-2, a task of 30 minutes or less gains another +6.

- At user energy 1-2, a nonurgent task at least 60 minutes with high requirement loses another -10.

- At user energy 4-5, a high-requirement task that is planned or goal-linked gains another +6.

Additional user-energy fit

Context	Adjustment
Energy 1-2: task <=30 minutes	+8
Energy 1-2: life-admin task	+7
Energy 1-2: low_energy/admin/recovery/maintenance micro-action	+8
Energy 1-2: rest action	+6
Energy 4: planned / goal-linked / high-priority task	+7 / +6 / +6
Energy 5: planned / goal-linked / high-priority task	+10 / +9 / +9
Energy 4 / 5: creative task	+4 / +7
Energy 4 / 5: focus or creative micro-action	+5 / +8

Low-energy preference score

Preference	Active only in low energy/state context	Adjustment
short_tasks	Task <=30 minutes	+9
admin	Life-admin task or admin micro-action	+9
recovery	Recovery micro-action or rest	+9
health	Movement/recovery/maintenance micro-action or water/move/rest	+8
critical_only	Pending tasks plus noncritical lifestyle/micro candidate	-16

Final state and time fit

- stateFit starts with the preselection state adjustment multiplied by 3.

- An eligible micro-action state tag adds +4; a mismatch would subtract 6, though mismatches are normally filtered earlier.

- CLEAR task: planned +7, goal-linked +5, and >=45 minutes with user energy >=4 adds +5.

- STRESSED task: <=30 minutes +7; >=60 minutes noncritical -8.

- RESTLESS task: health-energy category +7; <=25 minutes +4.

- OVERLOADED task: low task energy +8; <=30 minutes +7; >=60 minutes noncritical -12.

- A task fitting the available calendar window adds +6; an oversized noncritical task loses 14; critical receives 0 instead of the penalty.

- focus_25 loses 8 when under 25 minutes are available; focus_10 gains 3 when at least ten minutes are available.

Cooldown and variety

Action/rule	Default dismissal cooldown
`critical_overdue_task`	20 minutes
`closest_deadline_task`	30 minutes
`review_day`	2 hours
`complete_habit`	45 minutes
`start_task`	45 minutes
`log_water/log_meal/move/rest`	3 hours
`acknowledge_block`	30 minutes
`micro_action`	Template cooldownHours, otherwise 2 hours
`Other`	45 minutes

- A cooling candidate receives -1000 and is excluded from the available pool; debug still retains scored cooling candidates.

- Dismissal stores dismissedAt, cooldownUntil, and cooldownIdentity in contextSnapshot.

- If multiple dismissals share an identity, the latest expiration wins.

- The last sixteen non-dismissed suggestion identities within 24 hours drive variety: -7 per repeat, capped at -18.

- Hard-rule candidates do not receive a variety penalty.

Task category normalization and scoring

Raw standard/alias	Normalized group	Engine effect
School, Work, School Work	`school_work`	Matches school_work Needs Profile
Health, Movement, Exercise, Physical, Health Energy	`health_energy`	Matches health_energy; restless health fit
Admin, Maintenance, Personal, Life Admin	`life_admin`	Matches life_admin; low-energy admin fit
Creative, Creative Projects	`creative_projects`	Matches creative_projects; high-energy creative fit
Social	`social`	Recognized but neutral/grouping only
Any other custom value	`unknown`	Neutral category scoring

Normalization trims, lowercases, converts underscores, ampersands, slashes, and hyphens to spaces, and collapses repeated whitespace. User-visible labels are not rewritten. Habit and Goal category selectors share the eight standard options and preserve custom values, but Habit.category and Goal.category do not influence NOW_WHAT?.

Recommendation deck construction

- Previously shown suggestions are marked ignored before generation.

- Available and cooling candidates are scored separately; state-only rules are removed from the normal actionable pool.

- Deck state is no_context when hasUserContext is false, ready when an actionable candidate exists, cooldown when only cooling candidates remain, and empty when clear_state is the best fallback.

- A hard-rule primary has no alternatives. Otherwise, the engine first selects up to two alternatives with different action types, then fills remaining slots by identity.

- The primary suggestion is persisted unless it is a state-only no-context/no-action/clear candidate.

- Critical support text keeps the earliest critical task visible when another action wins.

Accept, dismiss, and alternative behavior

Action type	Accept behavior
`start_task`	Marks suggestion accepted, navigates to Tasks, and shows a toast; no in-progress task state exists
`complete_habit`	Writes completion history and habit_completed log, completes suggestion
`log_water/log_meal/rest/move/begin_focus`	Opens Quick Log
`review_day`	Navigates to Review; yesterday command triggers yesterday navigation
`plan_tomorrow`	Navigates to Dashboard
`acknowledge_block`	Shows acknowledgement toast; does not mutate block
`micro_action`	Routes actionIntent to modal, module, profile, Quick Log, or dashboard
`dismiss`	Stores cooldown metadata, then generates a new deck
`select alternative`	Promotes alternative, moves former primary into alternatives, updates selected debug identity

Explanation and debug visibility

- Reasons begin with a rule-specific factual sentence.

- Energy explanation is appended only when combined user/task energy fit is at least 8.

- Needs Profile explanation appears when needsScore is at least 7.

- Fits available time appears for a start_task with time score at least 5.

- Only the first two generated explanation fragments are appended to keep the card concise.

- Shift+N toggles a read-only debug panel with context, deck identities, task energy matching, task category context, full score table, cooldown exclusions, hard-rule status, and selected suggestion details.

8. Data Lifecycle, Export, Import, and Integrity

Write paths

The application uses a mixture of focused data-service functions and direct Dexie writes inside components. Profile, calendar, review, reminders, Needs Profile, Daily Plan, export/import, and integrity logic are centralized. Task, habit, and some recommendation actions still write through the imported active db binding in UI components.

Operation	Primary writer	Atomicity/side effects
Profile bootstrap	`data/profile.ts`	Meta/profile database opening, legacy migration transaction, Settings/NeedsProfile ensure
Goal link reconciliation	`GoalModal`	One transaction across goals, tasks, habits
State Check save	`data/needsProfile.ts`	One transaction across activityLogs and needsProfiles
Calendar conversion	`data/calendar.ts`	One transaction across tasks and calendarBlocks
Backup restore	`data/exportImport.ts`	One transaction clears and repopulates all twelve stores
Task/habit completion	`Rows, pages, NowWhatCard, reminder tray`	Updates domain record plus ActivityLog; implementations are duplicated
Suggestion lifecycle	`engine/nowWhat.ts`	Suggestion status and contextSnapshot updates
Reminder generation	`data/reminders.ts`	Single-flight run with per-record writes and dedupe

Export format v3

Export produces a formatted JSON download named now_what_backup_<date>_<time>.json. It includes app marker, exportVersion, schemaVersion, exportedAt, exportable profile metadata, and arrays for every profile-scoped store.

```
stores = [
  tasks, habits, goals, calendarBlocks, activityLogs, dailyPlans,
  suggestions, dailyReviews, weeklyReviews, settings, reminders, needsProfiles
]
```

Backup validation

- Requires a top-level object with app exactly NOW_WHAT?.

- Accepts schema versions 1 through 3; older versions may omit reminders before v2 and needsProfiles before v3.

- Requires each applicable store to be an array.

- Requires every record to have a nonempty string id and rejects duplicate IDs within a store.

- Normalizes missing/legacy profile metadata to a legacy profile descriptor.

- Returns a per-store record-count preview before restore.

- Does not perform exhaustive per-field runtime schema validation before restore; the Integrity panel is the second line of defense.

Restore semantics

- Restore is replace-all, not merge: all twelve active profile stores are cleared and then repopulated inside one transaction.

- Imported Settings are rewritten to the currently active localProfileId so backup metadata cannot re-scope the active database.

- lastImportAt is touched after restore.

- A data-bearing legacy backup without tutorial metadata is marked tutorial-seen/dismissed to avoid false first-run onboarding.

- A data-bearing backup without NeedsProfile gets a default profile whose onboarding is dismissed rather than interrupting restored users.

- Suggestion history, including dismissed cooldowns, is exported and restored.

Clear and demo behavior

- Normal launch calls no automatic seeder; seedIfEmpty is intentionally empty.

- Clear all removes all twelve profile stores and reloads. Bootstrap recreates minimum Settings and NeedsProfile only.

- Clear suggestions removes only suggestion history and regenerates the engine.

- RESET DEMO DATA is an explicit Danger Zone action that clears all stores and calls reseedWithDemoData.

- The demo dataset contains six tasks and three activity logs, including an overdue critical example.

Integrity checks

Area	Checks	Auto-fix availability
NeedsProfile	Singleton presence, enum arrays, modes, interval 1-168, quiet times, dismiss count, schedule ordering	Can create missing default
Task	energyRequired enum and goal existence	Warning only
Habit	goal existence	Warning only
Goal	missing linked task/habit IDs and asymmetric reverse links	Can remove missing IDs
DailyPlan	missing task/habit/block IDs and duplicate arrays	Can deduplicate arrays
CalendarBlock	start/end presence/order and missing converted task IDs	Warning/error only
DailyReview	missing completed task, habit, and block references	Warning only
ActivityLog	required timestamps/type, missing related records, State Check fields, mood/energy integer 1-5	Error/warning only
Reminder	valid type/status/time and missing task/habit/block targets	Can cancel orphan reminders

9. PWA and Offline Architecture

Manifest

- Name NOW_WHAT?, short name NOW_WHAT, standalone display, portrait-primary orientation.

- Start URL and scope are relative to the app root.

- Background and theme colors use the application's near-black visual identity.

- 192x192 and 512x512 SVG PWA icons are present; the 512 icon is declared any maskable.

- The browser favicon uses the supplied local now_what-transparent_white.png asset. The non-square source was not substituted for square maskable PWA icons.

Service worker behavior

Phase/request	Behavior
Development host	Install skips shell caching; activate deletes caches, unregisters itself, and claims clients
Production install	Opens nowwhat-shell-v1.1.0, fetches index with reload, caches declared shell URLs, discovers same-origin assets and Google font CSS from index
Production activate	Deletes all caches except current cache name and claims clients
Navigation	Network-first; writes successful response as cached index; offline fallback to cached index or root
Static same-origin asset	Cache-first with background network refresh
Google font request	Cache-first; stylesheet parsing prefetches gstatic font files
Non-GET/cross-origin	Ignored except approved font origins

Install prompt state

AppShell captures beforeinstallprompt and provides installPrompt/onInstall props to Topbar, but the current Topbar implementation does not render an install button. Browser-native install surfaces can still be available when installability criteria are met. This is an implemented-state limitation, not a missing manifest or service worker.

Offline boundary

The application can load its shell and local records without a network after a successful production visit. External Google font acquisition may fail on the first fully offline visit if it was never cached. There is no cloud sync, push subscription, background sync, or server-side reminder delivery.

10. Quality Assurance and Verification

Focused v1.2 engine harness

scripts/v12-engine-qa.mjs starts Vite in SSR middleware mode and uses node:assert plus server rendering to test engine modules without a browser database. The current harness reports 77 assertions.

- Energy bands and LOW/MEDIUM/HIGH normalization.
- Pure no-context behavior without hydration or review fallback leakage.
- Low/high/neutral task-energy matching and overloaded short-task preference.
- Critical overdue, active calendar block, and due-soon hard-rule precedence.
- Needs Profile weighting, micro-action gating, template uniqueness, cooldown positivity, and needs coverage.
- User Energy Level integer validation, quiet hours, and repeated State Check dismissal backoff.
- Export/import store coverage, profile data detection, legacy schema acceptance, and onboarding compatibility.
- Debug panel semantic fields and read-only behavior.
- Integrity detection for invalid task energy and invalid user energy logs.
- Category normalization, standard mappings, unknown neutrality, option deduplication, legacy custom form preservation, and separation from task energy.

Latest verification state

Check	Result at source snapshot
`npm run qa:v1.2`	PASS - 77 assertions
`npm run typecheck`	PASS - tsc --noEmit
`npm run build`	PASS - 109 modules transformed
`Production bundle`	CSS 73.78 kB; JS 562.09 kB before gzip
`Known build warning`	Main JS chunk exceeds Vite's 500 kB advisory threshold
`Category browser QA`	Desktop and 390px mobile controls; custom path; no console errors
`Favicon/PWA assets`	New favicon HTTP 200; manifest and service-worker paths valid

Important test gaps

- There is no broad component/unit test suite for every CRUD surface.
- IndexedDB migration and full file-upload restore are not exercised by the Vite SSR harness.
- Service-worker offline behavior is validated structurally but lacks a repeatable automated production browser test in the repository.
- Reminder scheduling is time-dependent and does not have fake-clock coverage.
- ICS parsing has no dedicated fixture suite for timezone and malformed-event edge cases.
- Accessibility is improved through semantic controls and focus states, but there is no automated axe-style audit.

11. Security, Privacy, and Reliability Posture

Privacy strengths

- No account identity, authentication token, remote API, telemetry SDK, or cloud database is required by application logic.
- Per-profile databases isolate normal domain records without adding profileId to every row.
- Export is initiated locally and produces a user-controlled JSON file.
- Browser notification permission is requested only through an explicit settings action.
- The engine is deterministic and does not transmit context for inference.

Security and data-loss considerations

- IndexedDB inherits the browser origin's storage and security model; clearing site data removes local records.
- Backups contain sensitive productivity and lifestyle context in plain JSON and are not encrypted by the app.
- Import is destructive replace-all. The Dexie transaction protects atomicity, but the app does not automatically create a pre-import rollback backup.
- Validation checks structure and IDs, not every field's runtime type. Integrity checks should run after imports from untrusted or manually edited files.
- Direct component writes make some cross-store invariants dependent on each UI path. Goal linking, State Check, calendar conversion, and restore use transactions; not every completion path is centralized.

12. Known Limitations and Technical Debt

Area	Current limitation	Impact
Versioning	Package/UI remain 1.1.0 while engine/debug refer to v1.2	Requires release coordination before a public v1.2 label
README	Still states that first load seeds demo data and lists the old SVG favicon	Documentation conflicts with current clean-first-run code
Profile UX	Meta architecture can hold profiles, but there is no profile switch/create UI	One resolved local profile is used per normal session
Database access	Many components import mutable global db directly	Harder to test and complicates live profile switching
Recurring tasks	RepeatRule is stored but future task generation is absent	Repeat selection does not materialize future instances
Calendar recurrence	RRULE is counted but not expanded	Recurring ICS events appear only as the imported base VEVENT
Calendar timezone	TZID wall time is treated as local; VTIMEZONE is not interpreted	Cross-timezone imports can be inaccurate
Calendar connectors	Google/Apple sources and UI placeholders exist, but no read-only API connector is implemented	ICS/manual are the working import paths
Reminders	Scheduler runs while app is open; no push/background scheduling	Closed-app reminders are not guaranteed
Install button	Install prompt state is captured but not rendered in Topbar	Browser-native install remains the available path
Import validation	No exhaustive field-level schema parser	Malformed field values may restore and then be reported by integrity checks
Category scoring	Social and unknown/custom task categories are intentionally neutral; Habit/Goal categories do not score	Custom semantics require a future explicit mapping policy
Action model	Starting a task navigates but TaskStatus has no in-progress value	Acceptance does not persist active execution state
Bundle size	Main production chunk is about 562 kB before gzip	Vite emits an advisory chunk-size warning
Testing	Focused engine harness, not full end-to-end coverage	CRUD, PWA, scheduler, and migration regressions need broader automation

Recommended extension order

- Correct README and release/version metadata so the documented first-run and engine versions match code.
- Centralize task/habit completion and activity-log writes in domain services to eliminate duplicated invariants.
- Add runtime backup schema validation and an optional automatic pre-import export.
- Add deterministic unit fixtures for ICS parsing, reminder clocks, Dexie migration, and export/import round trips.
- Implement recurrence materialization only after defining idempotency, timezone, and carry-over contracts.
- Surface the captured PWA install prompt and add a production offline browser test.
- Introduce explicit database/context injection before adding profile switching or sync.

13. Architecture Extension Map

Safe module boundaries for future work

Boundary	Keep stable	Extension point
Local Data Core	Domain interfaces, profile-per-database, export contract	Repositories/services over Dexie, runtime validators, migrations
NOW_WHAT?	Context -> candidates -> scores -> deck contract	New explainable rules and score components with QA fixtures
Calendar	CalendarBlock as read-only context	Google/Apple adapters normalized through normalizeExternalEventToBlock
Reminders	Reminder lifecycle and settings	Service-worker scheduling or native wrapper notification adapter
Quick Log	ActivityLog factual event contract	Additional bounded log types with review and integrity support
Review	Generated factual summaries	More transparent aggregates without motivational interpretation
PWA/mobile	Web-first routed React surface	Capacitor/native shell only after offline/profile/data flows remain stable

Design rule for future engine additions

EXPLAINABILITY TEST

A new recommendation factor should not ship unless it can be represented as stored context, a deterministic eligibility rule or numeric score component, a factual explanation fragment, a debug field, and a repeatable assertion.

Appendix A. Complete Micro-Action Library

This appendix is generated directly from MICRO_ACTIONS at commit 3d31a4b. Lists use comma-separated source values. Cooldowns are template-specific hours applied when the suggestion is dismissed.

Identity and command contract	User-facing body and eligibility tags
setup_add_task_today; command=ADD ONE TASK; category=setup; intent=add_task; cooldown=1h	Add one concrete task for today.; needs=school_work, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=no_tasks
setup_add_habit; command=ADD ONE HABIT; category=setup; intent=add_habit; cooldown=2h	Add one repeated action to track.; needs=habit_consistency, health_energy; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=no_habits
setup_add_calendar_block; command=ADD CALENDAR BLOCK; category=setup; intent=add_calendar_block; cooldown=2h	Add the next fixed commitment.; needs=school_work, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=no_calendar_blocks
setup_profile; command=CONFIGURE PROFILE; category=setup; intent=open_needs_profile; cooldown=6h	Set recommendation priorities and State Check preferences.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=needs_profile_incomplete
setup_capture_note; command=CAPTURE ONE NOTE; category=setup; intent=open_quick_log; cooldown=2h	Record one item before it is lost.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=always
planning_set_focus; command=SET PRIMARY FOCUS; category=planning; intent=plan_day; cooldown=3h	Set one primary focus for today.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=always
planning_next_30; command=PLAN NEXT 30 MINUTES; category=planning; intent=plan_day; cooldown=2h	Choose one bounded action for the next 30 minutes.; needs=school_work, life_admin, creative_projects; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral, high; conditions=always
planning_review_pending; command=REVIEW PENDING TASKS; category=planning; intent=review_tasks; cooldown=2h	Scan pending tasks and identify the next concrete item.; needs=school_work, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=pending_tasks
planning_tomorrow_first; command=PLAN TOMORROW FIRST; category=planning; intent=plan_day; cooldown=6h	Set tomorrow's first task.; needs=school_work, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=activity_exists
planning_upcoming_calendar; command=CHECK UPCOMING BLOCKS; category=planning; intent=open_calendar; cooldown=3h	Review the next calendar commitments.; needs=school_work, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=calendar_available
planning_carry_over; command=REVIEW UNFINISHED WORK; category=planning; intent=review_tasks; cooldown=4h	Choose one unfinished task to carry forward.; needs=school_work, life_admin, creative_projects; states=neutral, stressed, overloaded; capacity=normal, light, recovery; energy=low, neutral, high; conditions=pending_tasks
low_start_smallest; command=START SMALLEST PLANNED TASK; category=low_energy; intent=open_tasks; cooldown=2h	Open the task list and start the smallest planned item.; needs=school_work, life_admin, creative_projects; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=planned_tasks
low_admin_item; command=OPEN LOW-EFFORT ADMIN; category=low_energy; intent=review_tasks; cooldown=2h	Review pending tasks for one low-effort admin item.; needs=school_work, life_admin; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=pending_tasks
low_break_next_step; command=DEFINE ONE NEXT STEP; category=low_energy; intent=open_tasks; cooldown=2h	Turn one large task into a concrete next step.; needs=school_work, life_admin, creative_projects; states=low, stressed, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=pending_tasks
low_choose_short; command=CHOOSE A SHORT TASK; category=low_energy; intent=review_tasks; cooldown=2h	Review pending work and choose a task under 30 minutes.; needs=school_work, life_admin, creative_projects; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=pending_tasks
low_review_list; command=SCAN TASK LIST; category=low_energy; intent=open_tasks; cooldown=2h	Review tasks without starting a long work block.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=pending_tasks
low_add_admin; command=ADD LIFE-ADMIN TASK; category=low_energy; intent=add_task; cooldown=3h	Capture one concrete life-admin item.; needs=life_admin; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=no_recent_task

Identity and command contract	User-facing body and eligibility tags
low_maintenance; command=CHOOSE MAINTENANCE ITEM; category=low_energy; intent=review_tasks; cooldown=3h	Open tasks and select one small maintenance action.; needs=life_admin, health_energy; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=pending_tasks
recovery_short_rest; command=TAKE SHORT REST; category=recovery; intent=log_rest; cooldown=3h	Open Quick Log and record a short intentional rest.; needs=health_energy, life_admin, habit_consistency; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=no_urgent_work
recovery_add_block; command=ADD RECOVERY BLOCK; category=recovery; intent=add_calendar_block; cooldown=4h	Add a bounded recovery block to the calendar.; needs=health_energy, life_admin; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=no_urgent_work
recovery_log_state; command=LOG CURRENT STATE; category=recovery; intent=log_state; cooldown=2h	Record current state and Energy Level.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=always
recovery_log_rest; command=LOG REST; category=recovery; intent=log_rest; cooldown=2h	Open Quick Log and record rest time.; needs=health_energy, habit_consistency; states=low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=always
recovery_choose_mode; command=REVIEW WORK OR RECOVERY; category=recovery; intent=review_tasks; cooldown=3h	Review pending work before choosing work or recovery.; needs=health_energy, school_work, life_admin; states=low, stressed, overloaded; capacity=normal, light, recovery; energy=low, neutral; conditions=pending_tasks
movement_short_reset; command=SHORT MOVEMENT RESET; category=movement; intent=log_movement; cooldown=2h	Open Quick Log and record a short movement reset.; needs=habit_consistency, health_energy; states=restless, low, stressed; capacity=normal, light, recovery; energy=low, neutral, high; conditions=no_urgent_work
movement_log; command=LOG MOVEMENT; category=movement; intent=log_movement; cooldown=2h	Record completed movement.; needs=habit_consistency, health_energy; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=always
movement_add_block; command=ADD MOVEMENT BLOCK; category=movement; intent=add_calendar_block; cooldown=4h	Add a short movement block to the calendar.; needs=health_energy, habit_consistency, life_admin; states=neutral, restless; capacity=normal, light, recovery; energy=neutral, high; conditions=no_urgent_work
movement_active_reset; command=TAKE ACTIVE RESET; category=movement; intent=log_movement; cooldown=2h	Use a brief active reset before the next work block.; needs=health_energy, habit_consistency, school_work; states=restless; capacity=normal, light; energy=neutral, high; conditions=no_urgent_work
admin_clear_small; command=CLEAR SMALL ADMIN ITEM; category=admin; intent=open_tasks; cooldown=2h	Open tasks and clear one small admin item.; needs=life_admin, school_work; states=clear, neutral, low, stressed, restless, overloaded; capacity=normal, light, recovery; energy=low, neutral, high; conditions=pending_tasks
admin_review_reminders; command=REVIEW REMINDER-BOUND WORK; category=admin; intent=review_tasks; cooldown=3h	Review tasks that need a time or reminder.; needs=life_admin, school_work, habit_consistency; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=pending_tasks
admin_calendar_to_task; command=REVIEW CALENDAR CONVERSION; category=admin; intent=open_calendar; cooldown=4h	Open calendar blocks and convert one actionable block if needed.; needs=life_admin, school_work; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=calendar_available
admin_capture_inbox; command=CAPTURE ADMIN TASK; category=admin; intent=add_task; cooldown=3h	Add one concrete admin task.; needs=life_admin; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=no_recent_task
admin_review_profile; command=REVIEW PROFILE SETTINGS; category=maintenance; intent=open_needs_profile; cooldown=12h	Check recommendation priorities and State Check timing.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=always
focus_10; command=START 10-MINUTE FOCUS; category=focus; intent=open_quick_log; cooldown=2h	Open Quick Log and begin a 10-minute focus block.; needs=school_work, creative_projects; states=clear, neutral, restless; capacity=deep, normal, light; energy=low, neutral, high; conditions=no_urgent_block
focus_25; command=START 25-MINUTE FOCUS; category=focus; intent=open_quick_log; cooldown=2h	Open Quick Log and begin a 25-minute focus block.; needs=school_work, creative_projects; states=clear, neutral, restless; capacity=deep, normal, light; energy=neutral, high; conditions=no_urgent_block
focus_deep_priority; command=OPEN HIGH-VALUE WORK; category=focus; intent=review_tasks; cooldown=2h	Review high-priority work for a deeper focus block.; needs=school_work, creative_projects; states=clear, neutral; capacity=deep, normal; energy=high; conditions=pending_tasks
focus_deadline; command=OPEN CLOSEST DEADLINE; category=focus; intent=review_tasks; cooldown=2h	Review the closest-deadline task.; needs=school_work, life_admin; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light; energy=low, neutral, high; conditions=pending_tasks
focus_resume_planned; command=RESUME PLANNED WORK; category=focus; intent=open_tasks; cooldown=2h	Open the planned task list.; needs=school_work, creative_projects, life_admin; states=clear, neutral, restless; capacity=deep, normal, light; energy=neutral, high; conditions=planned_tasks

Identity and command contract	User-facing body and eligibility tags
focus_log_session; command=LOG FOCUS SESSION; category=focus; intent=open_quick_log; cooldown=2h	Record a completed focus session.; needs=school_work, creative_projects, habit_consistency; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=always
creative_add_task; command=ADD CREATIVE TASK; category=creative; intent=add_task; cooldown=3h	Capture one concrete creative project task.; needs=creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=no_recent_task
creative_start_draft; command=OPEN CREATIVE DRAFT; category=creative; intent=open_tasks; cooldown=2h	Review creative tasks and start one draft.; needs=creative_projects; states=clear, neutral, restless; capacity=deep, normal, light; energy=neutral, high; conditions=pending_tasks
creative_review_goal; command=REVIEW PROJECT GOAL; category=creative; intent=review_tasks; cooldown=4h	Review tasks linked to an active project goal.; needs=creative_projects, school_work; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=goals_available
creative_link_goal; command=LINK WORK TO GOAL; category=creative; intent=open_tasks; cooldown=6h	Review tasks and connect one item to an active goal.; needs=creative_projects, school_work, life_admin; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=goals_available, pending_tasks
review_today_activity; command=REVIEW TODAY ACTIVITY; category=review; intent=open_review; cooldown=4h	Open Daily Review for today's recorded activity.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=neutral, stressed, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=review_available
review_complete_daily; command=COMPLETE DAILY REVIEW; category=review; intent=open_review; cooldown=6h	Open the available Daily Review.; needs=school_work, habit_consistency, health_energy, life_admin; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=review_available
review_weekly_patterns; command=REVIEW WEEKLY PATTERNS; category=review; intent=open_review; cooldown=12h	Open Review and inspect weekly activity patterns.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=activity_exists
review_goal_progress; command=REVIEW GOAL-LINKED WORK; category=review; intent=review_tasks; cooldown=6h	Review tasks connected to active goals.; needs=school_work, creative_projects, life_admin; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=goals_available
review_focus_balance; command=REVIEW FOCUS AND REST; category=review; intent=open_review; cooldown=8h	Open Review and inspect focus, rest, and distraction logs.; needs=health_energy, school_work, habit_consistency; states=neutral, low, stressed, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=activity_exists
maintenance_next_commitment; command=CHECK NEXT COMMITMENT; category=maintenance; intent=open_calendar; cooldown=3h	Open the calendar and check the next fixed commitment.; needs=life_admin, school_work, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=calendar_available
maintenance_habit_window; command=CHECK ACTIVE HABIT WINDOW; category=maintenance; intent=open_habits; cooldown=2h	Open habits and review the active window.; needs=habit_consistency, health_energy; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=active_habit_window
maintenance_log_water; command=LOG WATER; category=maintenance; intent=log_water; cooldown=2h	Open Quick Log and record water.; needs=health_energy, habit_consistency; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=user_context
maintenance_log_meal; command=LOG MEAL; category=maintenance; intent=open_quick_log; cooldown=3h	Open Quick Log and record a meal.; needs=health_energy, habit_consistency; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=user_context
maintenance_log_energy; command=LOG ENERGY LEVEL; category=maintenance; intent=open_quick_log; cooldown=2h	Open Quick Log and record current Energy Level.; needs=school_work, habit_consistency, health_energy, life_admin, creative_projects; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=user_context
maintenance_log_distraction; command=LOG DISTRACTION; category=maintenance; intent=open_quick_log; cooldown=3h	Open Quick Log and record distraction time.; needs=school_work, creative_projects, health_energy; states=neutral, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=user_context
maintenance_health_habit; command=ADD HEALTH HABIT; category=maintenance; intent=add_habit; cooldown=6h	Add one health-maintenance habit.; needs=health_energy, habit_consistency; states=clear, neutral, low, stressed, restless, overloaded; capacity=deep, normal, light, recovery; energy=low, neutral, high; conditions=no_habits

Appendix B. Core File Inventory

The companion ZIP includes the following committed core files plus this report and a SHA-256 package manifest. Historical output artifacts and preview logs are intentionally excluded from the new package.

Path	Purpose
.env.example	Project configuration, documentation, or dependency metadata.
.gitignore	Project configuration, documentation, or dependency metadata.
README.md	Project configuration, documentation, or dependency metadata.
components.json	Project configuration, documentation, or dependency metadata.
index.html	Project configuration, documentation, or dependency metadata.
package-lock.json	Project configuration, documentation, or dependency metadata.
package.json	Package version, scripts, dependencies.
public/favicon.svg	Static PWA or browser asset.
public/icon-192.svg	Static PWA or browser asset.
public/icon-512.svg	Static PWA or browser asset.
public/manifest.webmanifest	PWA metadata and install icons.
public/now_what-transparent_white.png	Static PWA or browser asset.
public/robots.txt	Static PWA or browser asset.
public/service-worker.js	Production shell caching and offline request strategy.
scripts/v12-engine-qa.mjs	77-assertion Vite SSR engine QA harness.
src/App.tsx	Application bootstrap, routes, global modals, shortcuts, service worker, reminder scheduler.
src/components/calendar/CalendarBlockModal.tsx	React component in the calendar feature surface.
src/components/calendar/CalendarBlockRow.tsx	React component in the calendar feature surface.
src/components/calendar/CalendarPage.tsx	React component in the calendar feature surface.
src/components/dashboard/ActivityFeed.tsx	React component in the dashboard feature surface.
src/components/dashboard/CalendarStrip.tsx	React component in the dashboard feature surface.
src/components/dashboard/Dashboard.tsx	React component in the dashboard feature surface.
src/components/dashboard/GoalStrip.tsx	React component in the dashboard feature surface.
src/components/dashboard/LifestyleStrip.tsx	React component in the dashboard feature surface.
src/components/dashboard/NowWhatCard.tsx	React component in the dashboard feature surface.
src/components/dashboard/ReviewPrompt.tsx	React component in the dashboard feature surface.
src/components/dashboard/TaskList.tsx	React component in the dashboard feature surface.
src/components/data/DangerZone.tsx	React component in the data feature surface.
src/components/data/DataPage.tsx	React component in the data feature surface.
src/components/data/ExportPanel.tsx	React component in the data feature surface.
src/components/data/ImportPanel.tsx	React component in the data feature surface.
src/components/data/IntegrityPanel.tsx	React component in the data feature surface.
src/components/data/SchemaPanel.tsx	React component in the data feature surface.
src/components/debug/EngineDebugPanel.tsx	React component in the debug feature surface.
src/components/goals/GoalModal.tsx	React component in the goals feature surface.
src/components/goals/GoalRow.tsx	React component in the goals feature surface.
src/components/goals/GoalsPage.tsx	React component in the goals feature surface.

Path	Purpose
src/components/habits/HabitModal.tsx	React component in the habits feature surface.
src/components/habits/HabitRow.tsx	React component in the habits feature surface.
src/components/habits/HabitsPage.tsx	React component in the habits feature surface.
src/components/layout/AppShell.tsx	React component in the layout feature surface.
src/components/layout/Sidebar.tsx	React component in the layout feature surface.
src/components/layout/Topbar.tsx	React component in the layout feature surface.
src/components/needs/NeedsOnboardingPrompt.tsx	React component in the needs feature surface.
src/components/needs/RecommendationProfilePanel.tsx	React component in the needs feature surface.
src/components/needs/StateCheckPrompt.tsx	React component in the needs feature surface.
src/components/planning/DailyPlanPanel.tsx	React component in the planning feature surface.
src/components/quicklog/QuickLogModal.tsx	React component in the quicklog feature surface.
src/components/reminders/InAppReminderTray.tsx	React component in the reminders feature surface.
src/components/reminders/ReminderRow.tsx	React component in the reminders feature surface.
src/components/reminders/ReminderSettingsPanel.tsx	React component in the reminders feature surface.
src/components/reminders/RemindersPage.tsx	React component in the reminders feature surface.
src/components/review/CarryOverReview.tsx	React component in the review feature surface.
src/components/review/DailyReviewPage.tsx	React component in the review feature surface.
src/components/review/DailyReviewPanel.tsx	React component in the review feature surface.
src/components/tasks/TaskModal.tsx	React component in the tasks feature surface.
src/components/tasks/TaskRow.tsx	React component in the tasks feature surface.
src/components/tasks/TasksPage.tsx	React component in the tasks feature surface.
src/components/timeline/Timeline.tsx	React component in the timeline feature surface.
src/components/tutorial/TutorialPage.tsx	React component in the tutorial feature surface.
src/components/tutorial/TutorialPrompt.tsx	React component in the tutorial feature surface.
src/components/tutorial/TutorialSection.tsx	React component in the tutorial feature surface.
src/components/ui/Modal.tsx	Reusable UI primitive or wrapper.
src/components/ui/Toast.tsx	Reusable UI primitive or wrapper.
src/components/ui/accordion.tsx	Reusable UI primitive or wrapper.
src/components/ui/alert-dialog.tsx	Reusable UI primitive or wrapper.
src/components/ui/alert.tsx	Reusable UI primitive or wrapper.
src/components/ui/aspect-ratio.tsx	Reusable UI primitive or wrapper.
src/components/ui/avatar.tsx	Reusable UI primitive or wrapper.
src/components/ui/badge.tsx	Reusable UI primitive or wrapper.
src/components/ui/breadcrumb.tsx	Reusable UI primitive or wrapper.
src/components/ui/button-group.tsx	Reusable UI primitive or wrapper.
src/components/ui/button.tsx	Reusable UI primitive or wrapper.
src/components/ui/calendar.tsx	Reusable UI primitive or wrapper.
src/components/ui/card.tsx	Reusable UI primitive or wrapper.
src/components/ui/carousel.tsx	Reusable UI primitive or wrapper.
src/components/ui/chart.tsx	Reusable UI primitive or wrapper.
src/components/ui/checkbox.tsx	Reusable UI primitive or wrapper.
src/components/ui/collapsible.tsx	Reusable UI primitive or wrapper.

Path	Purpose
src/components/ui/command.tsx	Reusable UI primitive or wrapper.
src/components/ui/context-menu.tsx	Reusable UI primitive or wrapper.
src/components/ui/dialog.tsx	Reusable UI primitive or wrapper.
src/components/ui/drawer.tsx	Reusable UI primitive or wrapper.
src/components/ui/dropdown-menu.tsx	Reusable UI primitive or wrapper.
src/components/ui/empty.tsx	Reusable UI primitive or wrapper.
src/components/ui/field.tsx	Reusable UI primitive or wrapper.
src/components/ui/form.tsx	Reusable UI primitive or wrapper.
src/components/ui/hover-card.tsx	Reusable UI primitive or wrapper.
src/components/ui/input-group.tsx	Reusable UI primitive or wrapper.
src/components/ui/input-otp.tsx	Reusable UI primitive or wrapper.
src/components/ui/input.tsx	Reusable UI primitive or wrapper.
src/components/ui/item.tsx	Reusable UI primitive or wrapper.
src/components/ui/kbd.tsx	Reusable UI primitive or wrapper.
src/components/ui/label.tsx	Reusable UI primitive or wrapper.
src/components/ui/menubar.tsx	Reusable UI primitive or wrapper.
src/components/ui/navigation-menu.tsx	Reusable UI primitive or wrapper.
src/components/ui/pagination.tsx	Reusable UI primitive or wrapper.
src/components/ui/popover.tsx	Reusable UI primitive or wrapper.
src/components/ui/progress.tsx	Reusable UI primitive or wrapper.
src/components/ui/radio-group.tsx	Reusable UI primitive or wrapper.
src/components/ui/resizable.tsx	Reusable UI primitive or wrapper.
src/components/ui/scroll-area.tsx	Reusable UI primitive or wrapper.
src/components/ui/select.tsx	Reusable UI primitive or wrapper.
src/components/ui/separator.tsx	Reusable UI primitive or wrapper.
src/components/ui/sheet.tsx	Reusable UI primitive or wrapper.
src/components/ui/sidebar.tsx	Reusable UI primitive or wrapper.
src/components/ui/skeleton.tsx	Reusable UI primitive or wrapper.
src/components/ui/slider.tsx	Reusable UI primitive or wrapper.
src/components/ui/sonner.tsx	Reusable UI primitive or wrapper.
src/components/ui/spinner.tsx	Reusable UI primitive or wrapper.
src/components/ui/switch.tsx	Reusable UI primitive or wrapper.
src/components/ui/table.tsx	Reusable UI primitive or wrapper.
src/components/ui/tabs.tsx	Reusable UI primitive or wrapper.
src/components/ui/textarea.tsx	Reusable UI primitive or wrapper.
src/components/ui/toaster.tsx	Reusable UI primitive or wrapper.
src/components/ui/toggle-group.tsx	Reusable UI primitive or wrapper.
src/components/ui/toggle.tsx	Reusable UI primitive or wrapper.
src/components/ui/tooltip.tsx	Reusable UI primitive or wrapper.
src/constants/categories.ts	Application source module.
src/data/calendar.ts	Data-layer service or persistence helper.
src/data/db.ts	Dexie profile schema versions and active database binding.

Path	Purpose
src/data/exportImport.ts	Backup v3 export, validation, replace-all restore, clear operations.
src/data/ics.ts	Data-layer service or persistence helper.
src/data/integrity.ts	Cross-store and value integrity checks with selected fixes.
src/data/needsProfile.ts	Data-layer service or persistence helper.
src/data/profile.ts	Local profile metadata, profile database bootstrap, legacy migration.
src/data/reminders.ts	Data-layer service or persistence helper.
src/data/review.ts	Data-layer service or persistence helper.
src/data/seed.ts	Data-layer service or persistence helper.
src/data/tutorial.ts	Data-layer service or persistence helper.
src/engine/dailyPlan.ts	Rule engine or planning module.
src/engine/microActions.ts	54-template micro-action library.
src/engine/nowWhat.ts	Context, rules, state fit, cooldowns, deck, suggestion lifecycle.
src/engine/recommendationScoring.ts	v1.2 score components, hard rules, ranking, support text.
src/hooks/use-mobile.tsx	Application source module.
src/hooks/use-toast.ts	Application source module.
src/index.css	Global tokens, HUD styling, interaction states, responsive layouts.
src/lib/utils.ts	Application source module.
src/main.tsx	Application source module.
src/pages/not-found.tsx	Application source module.
src/types/index.ts	Shared domain and engine contracts.
src/utils/dateUtils.ts	Application source module.
tsconfig.json	Project configuration, documentation, or dependency metadata.
vite.config.ts	Project configuration, documentation, or dependency metadata.

Appendix C. Package Integrity and Handoff Notes

Source fingerprint

```
branch=feature/needs-profile-data-model
commit=3d31a4b
coreFileCount=144
aggregateSourceSha256=c3d51682c985e7c2404e94fcf61e9898c1a2d411dc56ac5a3ca6ae4cd64e7dad
```

LLM handoff guidance

- Treat src/types/index.ts as the domain contract and src/data/db.ts as the IndexedDB index/migration contract.

- Trace recommendations from getCurrentContext to getCandidateActions to scoreAndRankCandidates to generateRecommendationDeckWithDebugInfo.

- Do not collapse user Energy Level 1-5 into Task.energyRequired low/medium/high.

- Do not infer that Habit or Goal category affects recommendation scoring; only Task category currently maps to Needs Profile groups.

- Do not treat README claims as authoritative where they conflict with source; the clean-first-run seeder and current favicon are examples.

- Run npm run qa:v1.2, npm run typecheck, and npm run build after engine or data-layer changes.

- Preserve profile-specific storage, import compatibility, explicit manual demo reset, and deterministic explanations.

Glossary

Term	Meaning in NOW_WHAT?
`Candidate`	A potential action before final ranking and persistence.
`Suggestion`	Persisted user-facing representation of a candidate.
`Recommendation Deck`	Primary suggestion plus up to two alternatives and support/debug metadata.
`Hard rule`	Candidate with dominant hard score and no alternatives when primary.
`Needs Profile`	Local deterministic preference configuration, not an account profile.
`State Check`	Optional operational state plus required user Energy Level 1-5.
`Task energy`	Low/medium/high effort requirement stored on a task.
`User energy`	Latest validated numeric 1-5 ActivityLog signal.
`Micro-action`	Bounded non-task template with eligibility tags, intent, and cooldown.
`Cooldown identity`	Stable semantic key used to suppress a dismissed action temporarily.
`Profile database`	nowwhat_profile_<id>, containing all ordinary user records.
`Meta database`	nowwhat_meta, containing LocalProfile records and active/migration keys.

END OF REPORT

This document describes the complete implemented architecture at commit 3d31a4b. The companion ZIP is the source of truth for exact code behavior.